

AI-Powered Financial Dream & Goal Planner

Advanced Edition • Predict • Plan • Invest • Achieve

PROJECT CHALLENGE

Build a local end-to-end financial planning assistant for freshers. The system combines **Machine Learning, data preprocessing, model comparison, deterministic financial calculations, API development, feasibility analysis, RAG and Agentic AI** without requiring paid APIs.

ML Train + compare models	PLAN 3 future goals	INFLATION 6% per year	AGENT Tools + RAG	LOCAL No paid API
-------------------------------------	-------------------------------	---------------------------------	-----------------------------	-----------------------------

1 Project Scenario

The target users are final-year/fresher students who are starting their careers and have future dreams. The application should answer: *How much will my future goals cost, how much should I invest every month, and what broad investment category may fit each goal duration?*

2 User Inputs

Input	Example
Name	Rahul
Age	22
City	Bangalore
Education	B.Tech
Job Role	Software Engineer
Marriage	After 5 years
New Car	After 4 years
New Home	After 10 years
Savings / Investment Target	20% of income

Fresher Rule: Experience is not an input and must not be used as a salary-model feature.

3 Mandatory Machine Learning Work

- Use the provided salary dataset as training data.
- Perform preprocessing for categorical variables such as City, Education and Job Role.
- Train at least **three regression models**, e.g. Linear Regression, Decision Tree Regressor and Random Forest Regressor.
- Evaluate the models using suitable metrics such as MAE and R^2 .
- Compare the models and select the best-performing model using a documented rule.
- Save the selected trained model and use it in the application.
- Experience must not be included because all users are freshers.

4 Financial Planning Engine

- **Future Goal Cost:** Estimate future Marriage, Car and Home cost using the provided city/goal dataset.
- **Inflation:** Use a fixed project inflation rate of **6% per year**.
- **Investment Requirement:** Calculate approximate monthly investment needed for each goal using a clearly documented expected-return assumption.
- **Goal-wise Planning:** Calculate Marriage, Car and Home separately because their timelines can differ.
- **Total Requirement:** Calculate the combined monthly investment requirement for all selected goals.

Feasibility Analysis — Mandatory

The system must identify whether the user's selected saving percentage is sufficient for the planned goals. It should show the gap rather than hiding it.

Example

Home goal requires: ■35,000/month
 Available investment capacity: ■22,000/month
 Shortfall: ■13,000/month

The system should suggest ways to reduce the gap, such as **increasing the timeline, increasing the saving percentage, or adjusting the goal amount.**

- Classify each goal as **Achievable, Challenging** or **Highly Challenging**.
- Show the monthly shortfall/surplus.
- Allow the user to test a changed timeline or saving percentage and recalculate the plan.

6 Investment Category Recommendation

The system should recommend only broad educational categories based on the time horizon. Students must define and document their own rule set.

- **Short-term goal:** lower-volatility / capital-preservation-oriented category.
- **Medium-term goal:** diversified balanced category.
- **Long-term goal:** diversified long-term growth-oriented category.
- Do not recommend individual stocks, guarantee returns, or present the system as professional financial advice.

7 Agentic AI — Mandatory Advanced Component

Build a local Agent that understands the user's request and decides which tool(s) are required. The Agent must not perform deterministic financial calculations itself.

Tool	Responsibility
Salary Prediction Tool	Calls the student's selected ML model
Future Cost Tool	Calculates future goal cost using 0.06% annual inflation
Investment Calculator Tool	Calculates required monthly investment
Feasibility Tool	Calculates capacity, shortfall/surplus and goal status
Recommendation Tool	Selects broad investment category using documented rules

8 RAG Knowledge Base — Mandatory Advanced Component

Create a small local knowledge base containing project-approved financial-planning information. Use RAG to retrieve relevant information for explanatory questions instead of allowing the model to invent facts.

- Create at least 3 local documents, for example: **financial_guidelines.txt**, **investment_categories.txt**, and **goal_planning_rules.txt**.
- Chunk the documents and create embeddings.
- Store embeddings in a local vector store.
- Retrieve relevant chunks for user questions.
- Return an answer grounded in the retrieved content.
- If the answer is not present in the knowledge base, the system should say that the information is unavailable rather than hallucinating.

9 Natural-Language User Interaction

The Agent should understand requests such as: *"I am 22, live in Bangalore, expect to start as a software engineer, want to marry in 5 years, buy a car in 4 years and a home in 10 years. I want to save 20% of my income. What should my plan look like?"*

The system should extract the relevant information, call the appropriate tools, perform the calculations, retrieve supporting knowledge through RAG when needed, and return a clear final answer.

10 API / Application Requirement

- Expose the application through a local API; FastAPI is recommended.
- The application must run on the student's laptop.
- No paid API is required.
- If an LLM is used, students may use a local runtime such as Ollama.
- Keep calculations and business rules in normal Python/tool functions.

11 Testing & Hallucination Control

- Minimum **15 test cases** covering valid, invalid, boundary and end-to-end inputs.
- Test unknown cities, unknown education/job roles and invalid timelines.
- Test saving percentage below 0% and above 100%.
- Test a goal whose required investment exceeds available capacity and verify the shortfall is shown.
- Ask questions outside the RAG knowledge base and verify the Agent does not invent an answer.
- Test prompt-injection attempts and verify that system instructions/tools are not bypassed.
- Record expected output and actual output for each test.

12 Required Deliverables

- Complete source-code folder.
- **Updated salary training dataset** and city/goal cost dataset.
- Trained model(s) and selected final model.
- Preprocessing/model-comparison code.
- API/application code.
- Local RAG knowledge-base documents, embeddings/vector-store setup and retrieval code.
- Agent + tool definitions.
- README with installation and execution steps.
- DFD/architecture diagram prepared by the student.
- Minimum 15 test cases with results.
- Short project report explaining assumptions and algorithms.
- Demonstration video showing the complete system running on the student's laptop.
- Final live demonstration and viva.

13 Viva Questions

- Why is salary prediction a Regression problem?
- Why did you compare multiple regression models?
- Why was the final model selected?
- How did you encode City, Education and Job Role?
- Why is Experience excluded?
- How is future cost calculated?
- Why is the project inflation rate 0.06% per year?
- How is monthly investment calculated?
- How do you calculate the shortfall in the feasibility analysis?
- How does changing the goal timeline affect the plan?
- Why should financial calculations be implemented as tools instead of relying on an LLM?
- What is RAG and why is it used here?
- What happens when RAG cannot find relevant information?
- How does your Agent decide which tool to call?
- How did you test hallucination and prompt injection?

14 Suggested Marking Scheme — 100 Marks

Component	Marks
Problem understanding & input design	5
Dataset & preprocessing	10
ML model training + model comparison	20
Financial calculations	15
Feasibility analysis	10
Investment category logic	5

API/application	10
Agentic AI + tools	10
RAG + grounding	5
Testing + safety	5
Demo + viva	5

FINAL PROJECT RULES

1. All students are treated as freshers; Experience is not required and must not be used.
2. Use **6% inflation per year** for future goal-cost calculations.
3. Financial calculations must be deterministic and reproducible; do not ask an LLM to invent numeric results.
4. The project is an educational simulation, not professional financial advice. Returns are not guaranteed.
5. No paid API is required; the complete project must be demonstrable locally.